

Rapport

Projet : Réorganisation d'un réseau de fibres optiques

Participants

- Bogush Ekaterina
- Combo Lyna

Sujet

Ce projet a pour but de réorganiser le réseau de fibres optiques d'une agglomération. Il est composé de deux parties :

1. Reconstituer un réseau composé des chaînes et des points fournies.
2. Optimiser un réseau en minimisant la longueur totale des chaînes.

Partie 1 - Reconstitution Réseau

Dans cette partie, nous reconstituons le réseau à l'aide de trois structures des données différentes:

1. Liste simplement chaînée

Les fichiers avec les fonctions associées : `Reseau.h`, `Reseau.c`. La fonction principale est `reconstitueReseauListe`. Elle permet de reconstituer un réseau à l'aide de la liste simplement chaînée (en utilisant directement l'attribut `noeuds` qui représente la liste chaînée de la structure `Reseau`) à partir des chaînes.

Ces fichiers contiennent aussi la définition des structures du Réseau et les fonctions permettant de le manipuler.

2. Table de hachage

Les fichiers avec des fonctions associées : `Hachage.h`, `Hachage.c`. La fonction principale est `reconstitueReseauHachage`. Elle permet de reconstituer un réseau à l'aide d'un table de hachage à partir des chaînes. Il existe 4 fonctions auxiliaires qui permettent de manipuler la table de hachage. `clef` (génère le clé d'un point), `hash` (la fonction de hachage donnée dans l'énoncé), `creer_tab` (crée et initialise la table de hachage), `insérer` (insère un élément dans un bon endroit dans la table de hachage).

3. Arbre quaternaire.

La racine représente le centre de la carte. Chaque nœud interne représente un point intercardinal (nord-est, nord-ouest, sud-est, sud-ouest). Les feuilles représentent un point (un client ou un concentrateur).

Les fichiers avec les fonctions associées : `ArbreQuat.h`, `ArbreQuat.c`.

La fonction principale est `reconstitueReseauArbre`. Elle permet de reconstituer un réseau à l'aide d'un arbre quaternaire à partir des chaînes. Il existe deux fonctions auxiliaires qui permettent de construire un arbre quaternaire : `creerArbreQuat` (crée un arbre vide) et `insérerNoeudArbre` (insère d'un point dans un bon endroit).

Le `main` pour tester les ces 3 structures de données se trouve dans le fichier `ReconstitueReseau.c`.

Partie 2

Dans cette partie on utilise le graphe non-orienté et non pondéré. Les fichiers avec les fonctions associées : `Graphe.c` , `Graphe.h` . Le main de graphe se trouve dans le fichier `GrapheMain.c` .

Pour construire le graphe la fonction principale est `creerGraphe` qui fait appel à 2 fonctions auxiliaires `majVoisins` (ajout des arêtes) `majCommodites` (ajout des commodités).

Pour trouver le plus court chemin, on utilise la fonction `trouverCheminMin` . Elle utilise le BFS et fait appel aux 2 fonctions auxiliaires : `enfilerSuccesseurs` (MAJ des pères, des distances et des visites) : `creerPlusCourtChemin` (créer un chemin minimal).

Les structures auxiliaires

Nous avons codé nous-mêmes des structures supplémentaires pour la partie avec des graphes.

- File: Fichiers `utls/File.c` et `utls/File.h` .
- Liste chaînée des entiers: Fichiers `utls/ListeEntiers.c` et `utls/ListeEntiers.h` .
- Matrices des entiers: Fichiers `utls/MatriceEntiers.c` et `utls/MatriceEntiers.h` .

Exercice 4. Table de hachage

Question 2

La fonction de la génération d'une clé donne les valeurs deux à deux distinctes pour

$1 \leq x, y \leq 10$. Nous concluons que la fonction est appropriée parce qu'elle permettra d'éviter au mieux les collisions dans la table de hachage.

Exercice 6. Comparaison des structures

Question 1.

Les résultats des tests de la de la reconstitution d'un réseau sur les petites données (cf.

`graphes/instances_analyse.txt` pour des résultats du test).

instances_analyse

Aa Instance	# Liste	# Hash_2	# Hash_5	# Hash_10	# Arbre
data/00150_ch.cha	0.000263	0.000159	0.000157	0.000171	0.000257
data/00417_fl.cha	0.0023	0.000699	0.00066	0.000678	0.001154
data/00100_USA-road-d-NY-1-100.cha	0.000038	0.00004	0.00004	0.000043	0.000083
data/00076_eil.cha	0.000064	0.000056	0.000057	0.000061	0.000086
data/00144_pr.cha	0.000203	0.000118	0.000117	0.000122	0.000208
data/00014_burma.cha	0.000009	0.000013	0.000011	0.00001	0.000014
data/00048_att.cha	0.000035	0.000039	0.00004	0.00004	0.000057
data/00052_berlin.cha	0.000047	0.000045	0.000049	0.000051	0.000069
data/00280_a.cha	0.000843	0.000296	0.000301	0.00032	0.000489
data/00783_rat.cha	0.015908	0.001933	0.001992	0.002061	0.003815
data/00575_rat.cha	0.006378	0.001067	0.00108	0.001129	0.00197

Aa Instance	# Liste	# Hash_2	# Hash_5	# Hash_10	# Arbre
data/00493_d.cha	0.064094	0.014891	0.014697	0.014778	0.017854

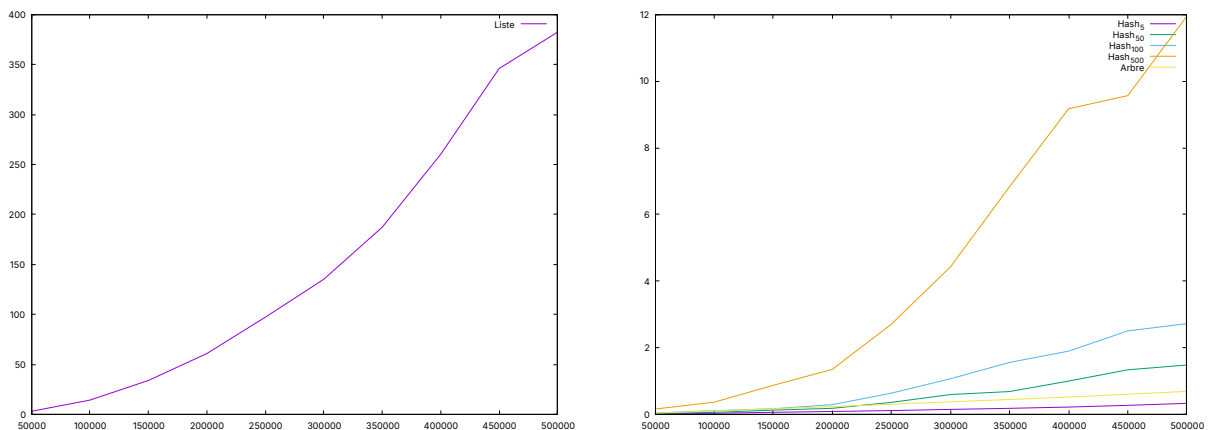
Pour cette question, nous varions la taille de la table de hachage de sorte que pour N un nombre total des points dans les chaînes, la taille de la table de hachage est $\frac{N}{2}, \frac{N}{5}, \frac{N}{10}$ (dans le tableau et le fichier c'est noté comme $Hash_diviseur$).

Nous remarquons que sur le petit nombre des données, la liste chaînée est plus performante que l'arbre quaternaire. Par contre, dès que le nombre des données augmente, la performance de l'arbre surpasse complètement celle de la liste.

La table de hachage de telles tailles est plus performante que la liste et l'arbre. Nous remarquons que plus elle est grande, plus l'algorithme de la reconstitution du réseau s'exécute rapidement, ce qui est logique car la complexité de recherche d'un nœud est en $O(\alpha)$ avec α : facteur de charge. Plus la table est grande, plus le facteur de charge est petit. Cependant, puisqu'ici nous travaillons sur un petit nombre des données, nous pouvons nous attendre à avoir beaucoup de cases libres si la table est trop grande.

Question 4

Cette fois, nous testons nos trois fonctions de reconstitution d'un réseau sur le grand nombre des données (génération aléatoire). Les graphes représentent le temps de reconstitution du réseau (en secondes) en fonction du nombre total de chaînes (comptage est effectué avec les répétitions éventuelles, comme demandé dans la fonction `comptePointsTotal`).



Soient N : un nombre des points dans la chaîne, M : un nombre des chaînes.

Nous pouvons classer la performance des structures des données de façon suivante (ordre décroissant) :

1. Table de hachage. Nous faisons varier sa taille entre $\frac{N * M}{5}, \frac{N * M}{50}, \frac{N * M}{100}, \frac{N * M}{500}$. Dans le cadre de cette question, nous générons les données aléatoirement. Pour chaque taille, le facteur de charge est respectivement 5, 50, 100, 500. Il est noté sur la légende du graphique à droite comme $Hash_{facteur-de-charge}$.

Remarque : Lorsque le facteur de charge est supérieur à 50, l'arbre quaternaire devient meilleur que la table de hachage.

2. Arbre quaternaire.

3. Liste simplement chaînée.

Complexités de la reconstitution du réseau :

- Liste simplement chaînée → complexité $O((N * M)^2)$
- Arbre quaternaire → complexité $O(h)$ avec h : hauteur de l'arbre. Nous pourrions l'améliorer avec les AVL et passer à une complexité en $O(\log_4(N * M))$.
- Table de hachage → Sous l'hypothèse de l'uniformité de la distribution des valeurs, complexité $O(\alpha)$ avec α : facteur de charge.

Si le facteur de charge $\alpha = 5$ et nous travaillons sur le grand nombre des données et avons $O(\alpha) \subset O(h)$. Si le facteur de charge $\alpha \geq 50$, nous avons $O(h) \subset O(\alpha)$.

Dans tous les cas, nous avons les inclusion $O(\alpha) \subset O((N * M)^2)$ et $O(h) \subset O((N * M)^2)$.

Nous les voyons bien sur les graphiques ci-dessus.

Exercice 7. Optimisation d'un réseau.

Question 1

Remarque sur notre implémentation de la consigne suivante :

Le numéro des noeuds dans le graphe doit correspondre à leur position dans le tableau de sommets.

Puisque c'est nous qui gérons les numéros des noeuds lors de la reconstitution du réseau (ils commencent à partir de 1 et augmentent de 1 à chaque fois), nous avons choisi de faire la correspondance suivante:

Numéro du noeud correspond à sa position dans un tableau avec décalage à -1. Cela permet de retrouver les voisins et les commodités en complexité de $O(1)$.

Exemple : le noeud de numéro 1 se trouve à l'indice 0 dans un tableau de sommets

Sinon, on pourrait allouer un tableau supplémentaire des associations des numéros des noeuds dans le Réseau et des numéros des sommets dans G pour préserver la complexité de $O(1)$.

Question 3

Pour stocker l'arborescence des chemins issus de u à v , nous pouvons utiliser 2 tableaux :

1. Un tableau pour stocker des longueurs des chemins à partir de u . Dans le code c'est le tableau `distances` qui stocke des entiers.
2. Un tableau pour stocker les pères des noeuds. Dans le code c'est le tableau `peres` qui stocke le numéro des sommets-pères.

Question 4. Réorganisation du réseau

Suggestions:

- C'est un graphe non-orienté, alors matrice associée est symétrique. Nous pourrions stocker au moins la moitié de la matrice (par exemple la partie triangulaire inférieure) pour améliorer l'usage de la mémoire.

Nous introduisons le vocabulaire suivant:

- M : la matrice donnée à la question 7.4
- R_s : réseau reconstitué qui peut contenir une arête dont la charge est plus grande que γ
- R_{ns} : réseau reconstitué qui contient exactement les mêmes sommets et les mêmes commodités que R_s . Ainsi R_{ns} ne contient pas des arêtes dont la charge est plus grande que γ . De plus, R_{ns} a le même γ que R_s

Nous pouvons vérifier si le réseau R_s est saturé en utilisant la matrice M . S'il l'est, notre but est de réorganiser le réseau non-saturé R_{ns} .

Pour cela, nous pourrions implémenter l'algorithme suivant:

Initialisation : Nous copions les sommets et les commodités de R_s dans R_{ns} .

Nous utilisons le parcours en largeur pour construire les chaînes de longueur minimal.

→ pour chaque commodité $\{s1, s2\}$, avec $s1, s2$ les sommets de R_{ns}

→ parcourir une arête à partir de $s1$ une par une en utilisant le BFS

→ à l'aide de la matrice M , nous vérifions si l'arête $\{s_i, s_j\}$ est saturée, i.e. $M[s_i][s_j] > \gamma$.

Deux cas s'offrent à nous:

1. L'arête est saturée, alors elle n'est pas ajoutée dans la file associée au parcours BFS.

2. L'arête n'est pas saturée, alors elle est ajoutée dans la file associée au parcours BFS.

→ À la fin, nous arrivons au sommet $s2$. Nous avons donc construit la chaîne $s1 - \dots - s2$ non-saturée à l'aide de laquelle nous mettons à jour les voisins des noeuds associés aux sommets dans la chaîne construite dans le réseau R_{ns} .